

R Code Examples from WDDA Lecture 07 (Chapter 7)

WDDA – Data Management and Data Analysis

Prof. Dr. Ulrich Matter

2026-05-20

Contents

1	Introduction	1
2	Preparation	1
3	Bootstrap and Confidence Intervals	2
3.1	Simple Linear Regression	2
3.2	Visualising the Bootstrap Distributions	3
3.3	Multiple Regression	5
3.4	Visualising the Bootstrap Distributions for Multiple Regression	6
4	Inference and Significance Tests	7
4.1	t-Tests for Regression Coefficients	7
4.2	F-Statistic and Model Comparison	8
4.3	Prediction with Confidence and Prediction Intervals	9
4.4	Visualising Confidence and Prediction Intervals	10
5	Summary	11

1 Introduction

This document collects all R code examples from the slides of WDDA FS2026 Lecture 07 (Chapter 7). The focus is on inference in regression models, in particular on confidence intervals, significance tests, and predictions.

Statistical inference allows us to draw conclusions about the population from a sample. In regression analysis, we are interested in the uncertainty of the estimated coefficients and the reliability of our predictions.

Key concepts in this area are:

1. **Confidence intervals for regression coefficients:** How precise are our estimates?
2. **Bootstrap methods:** How can we simulate the distribution of the coefficients?
3. **Significance tests:** Are the observed effects statistically significant?
4. **Prediction intervals:** How reliable are our predictions?

2 Preparation

```
# Load packages
library(mosaic)      # for do(), resample()
```

```
library(ggplot2)    # for visualisations
library(readxl)     # for reading Excel files

# Load data
Advertising <- read_excel("data/WDDA_07.xlsx", sheet = "Advertising")

## tibble [200 x 4] (S3: tbl_df/tbl/data.frame)
## $ TV          : num [1:200] 230.1 44.5 17.2 151.5 180.8 ...
## $ radio       : num [1:200] 37.8 39.3 45.9 41.3 10.8 48.9 32.8 19.6 2.1 2.6 ...
## $ newspaper: num [1:200] 69.2 45.1 69.3 58.5 58.4 75 23.5 11.6 1 21.2 ...
## $ sales      : num [1:200] 22.1 10.4 9.3 18.5 12.9 7.2 11.8 13.2 4.8 10.6 ...
```

R explanation:

- The function `library()` loads an installed package into the current R session.
- The package `mosaic` provides functions for statistical simulations, in particular `do()` and `resample()`.
- The package `ggplot2` enables the creation of attractive visualisations.
- With `read_excel()` from the `readxl` package, Excel files can be imported.
- The function `str()` shows the structure of an object, including data types and dimensions.

Note on data paths: In R Markdown files, relative paths are interpreted from the location of the file, hence the path `"../data/WDDA_07.xlsx"`. In regular R scripts, the path would be specified relative to the working directory.

3 Bootstrap and Confidence Intervals

The bootstrap method is a resampling procedure that allows us to estimate the sampling distribution of a statistic without making assumptions about the underlying distribution.

3.1 Simple Linear Regression

```
# Simple regression model (TV)
mdl <- lm(sales ~ TV, data = Advertising)
summary(mdl)

##
## Call:
## lm(formula = sales ~ TV, data = Advertising)
##
## Residuals:
##      Min       1Q   Median       3Q      Max
## -8.3860 -1.9545 -0.1913  2.0671  7.2124
##
## Coefficients:
##              Estimate Std. Error t value Pr(>|t|)
## (Intercept)  7.032594   0.457843   15.36  <2e-16 ***
## TV           0.047537   0.002691   17.67  <2e-16 ***
## ---
## Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1
##
## Residual standard error: 3.259 on 198 degrees of freedom
## Multiple R-squared:  0.6119, Adjusted R-squared:  0.6099
## F-statistic: 312.1 on 1 and 198 DF,  p-value: < 2.2e-16
```

```
# Classical confidence interval
confint(mdl)

##              2.5 %      97.5 %
## (Intercept) 6.12971927 7.93546783
## TV          0.04223072 0.05284256

# Bootstrap simulation for coefficients
set.seed(123) # for reproducibility
simcoef <- do(5000) * coef(lm(sales ~ TV, data = resample(Advertising)))
CI_intercept <- quantile(simcoef$Intercept, c(0.025, 0.975))
CI_tv <- quantile(simcoef$TV, c(0.025, 0.975))
cat("Bootstrap 95% CI for intercept:", round(CI_intercept, 3), "\n")

## Bootstrap 95% CI for intercept: 6.391 7.697

cat("Bootstrap 95% CI for slope (TV):", round(CI_tv, 3), "\n\n")

## Bootstrap 95% CI for slope (TV): 0.042 0.053
```

Statistical explanation:

- **Confidence intervals** specify a range in which the true parameter value lies with a certain probability (e.g., 95%).
- The **bootstrap method** generates many samples by drawing with replacement from the original sample.
- For each bootstrap sample, the regression model is refitted and the coefficients are stored.
- The distribution of these coefficients approximates the sampling distribution of the estimators.
- The **95% confidence interval** is determined by the 2.5% and 97.5% quantiles of this distribution.
- Compared to the classical confidence interval (based on the t-distribution), the bootstrap method makes fewer assumptions about the distribution of the data.

R explanation:

- The function `lm()` (linear model) fits a linear regression model to the data.
- The function `confint()` computes confidence intervals for the coefficients of a regression model based on the t-distribution.
- The function `set.seed()` sets the seed for the random number generator in order to obtain reproducible results.
- The function `resample()` from the `mosaic` package draws a sample with replacement from the data.
- The function `do()` from the `mosaic` package repeats an expression multiple times and collects the results.
- The function `quantile()` computes quantiles of a distribution.

3.2 Visualising the Bootstrap Distributions

```
# Visualise the bootstrap distributions
par(mfrow = c(1, 3), cex.main = 1.8, cex.lab = 1.6, cex.axis = 1.4, mar = c(4, 4, 3, 1))

# Define common axis limits
tv_range <- range(simcoef$TV)
intercept_range <- range(simcoef$Intercept)
```

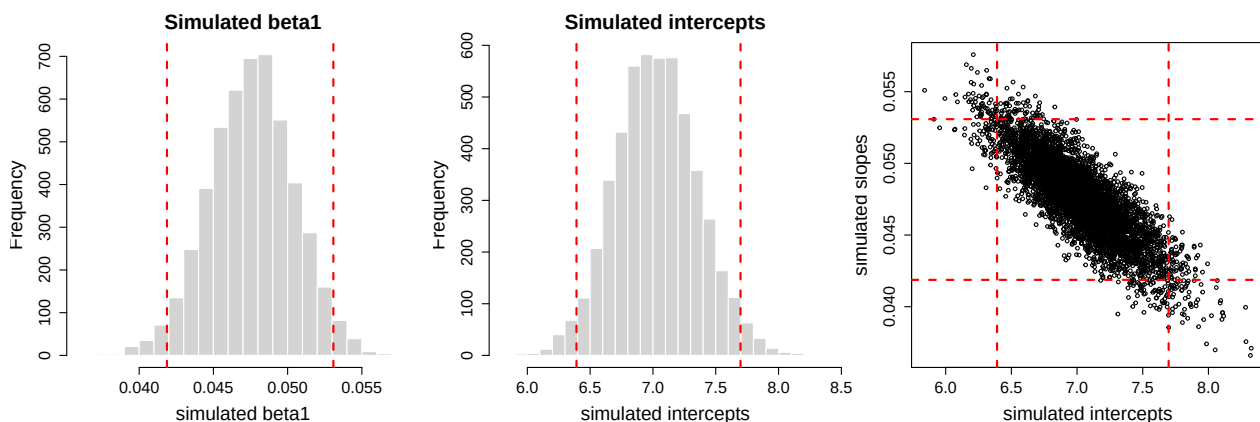
```

# Histogram for slope (beta1)
hist(simcoef$TV, main = "Simulated beta1", xlab = "simulated beta1",
     col = "lightgray", border = "white", breaks = 20,
     xaxt = "n") # No axis labelling at first
axis(1, at = axTicks(1)) # Explicit axis labelling
tv_ticks <- axTicks(1)
abline(v = quantile(simcoef$TV, c(0.025, 0.975)), col = "red", lty = 2, lwd = 2)

# Histogram for intercept (beta0)
hist(simcoef$Intercept, main = "Simulated intercepts", xlab = "simulated intercepts",
     col = "lightgray", border = "white", breaks = 20,
     xaxt = "n") # No axis labelling at first
axis(1, at = axTicks(1)) # Explicit axis labelling
intercept_ticks <- axTicks(1)
abline(v = quantile(simcoef$Intercept, c(0.025, 0.975)), col = "red", lty = 2, lwd = 2)

# Scatterplot of the coefficients
plot(simcoef$Intercept, simcoef$TV, pch = 1, cex = 0.7,
     xlab = "simulated intercepts", ylab = "simulated slopes",
     xaxt = "n", yaxt = "n") # No axis labelling at first
axis(1, at = intercept_ticks) # Same ticks as in the intercept histogram
axis(2, at = tv_ticks) # Same ticks as in the TV histogram
abline(v = quantile(simcoef$Intercept, c(0.025, 0.975)), col = "red", lty = 2, lwd = 2)
abline(h = quantile(simcoef$TV, c(0.025, 0.975)), col = "red", lty = 2, lwd = 2)

```



```
par(mfrow = c(1, 1))
```

Statistical explanation:

- The **histograms** show the distribution of the bootstrap estimates for the intercept and the slope (TV coefficient).
- The **red dashed lines** mark the bounds of the 95% confidence interval.
- The **scatterplot** shows the joint distribution of the two coefficients.
- The **negative correlation** between intercept and slope is typical for regression models and arises from the structure of the data and the estimation method.
- These visualisations help to understand the uncertainty of the estimates and the relationships between the coefficients.

R explanation:

- The function `par()` with the parameter `mfrow = c(1, 3)` divides the graphics window into 1 row and 3 columns.
- The parameters `cex.main`, `cex.lab`, and `cex.axis` in `par()` control the font size of titles, axis labels, and axis values.
- The function `hist()` creates a histogram.
- The function `abline()` with the parameter `v` adds vertical lines to the plot.
- The function `plot()` creates a scatterplot.

3.3 Multiple Regression

```
# Multiple regression (TV + radio)
md2 <- lm(sales ~ TV + radio, data = Advertising)
summary(md2)

##
## Call:
## lm(formula = sales ~ TV + radio, data = Advertising)
##
## Residuals:
##      Min       1Q   Median       3Q      Max
## -8.7977 -0.8752  0.2422  1.1708  2.8328
##
## Coefficients:
##              Estimate Std. Error t value Pr(>|t|)
## (Intercept)  2.92110    0.29449   9.919  <2e-16 ***
## TV           0.04575    0.00139  32.909  <2e-16 ***
## radio        0.18799    0.00804  23.382  <2e-16 ***
## ---
## Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1
##
## Residual standard error: 1.681 on 197 degrees of freedom
## Multiple R-squared:  0.8972, Adjusted R-squared:  0.8962
## F-statistic: 859.6 on 2 and 197 DF,  p-value: < 2.2e-16
```

```
confint(md2)
```

```
##              2.5 %      97.5 %
## (Intercept) 2.34034299 3.50185683
## TV          0.04301292 0.04849671
## radio       0.17213877 0.20384969
```

```
# Bootstrap for multiple regression
set.seed(456)
simcoef_multi <- do(5000) * coef(lm(sales ~ TV + radio, data = resample(Advertising)))
CI_tv_multi <- quantile(simcoef_multi$TV, c(0.025, 0.975))
CI_radio_multi <- quantile(simcoef_multi$radio, c(0.025, 0.975))
cat("95% CI for TV (multi):", round(CI_tv_multi, 3), "\n")
```

```
## 95% CI for TV (multi): 0.042 0.05
```

```
cat("95% CI for Radio (multi):", round(CI_radio_multi, 3), "\n\n")
```

```
## 95% CI for Radio (multi): 0.168 0.207
```

Statistical explanation:

- In **multiple regression**, confidence intervals are computed for each coefficient, taking into account the other predictors in the model.
- The interpretation of a confidence interval in multiple regression is: “Holding all other predictors constant, the true effect of the predictor lies within this interval with 95% probability.”
- The bootstrap method can also be applied to multiple regression models in order to estimate the distribution of the coefficients.
- Compared to simple regression, the confidence intervals for a predictor can change when other predictors are added to the model, especially when the predictors are correlated.

R explanation:

- The formula `sales ~ TV + radio` in `lm()` specifies that `sales` is the dependent variable and `TV` and `radio` are the independent variables.
- The expression `simcoef_multi$TV` extracts the column `TV` from the data frame `simcoef_multi`.

3.4 Visualising the Bootstrap Distributions for Multiple Regression

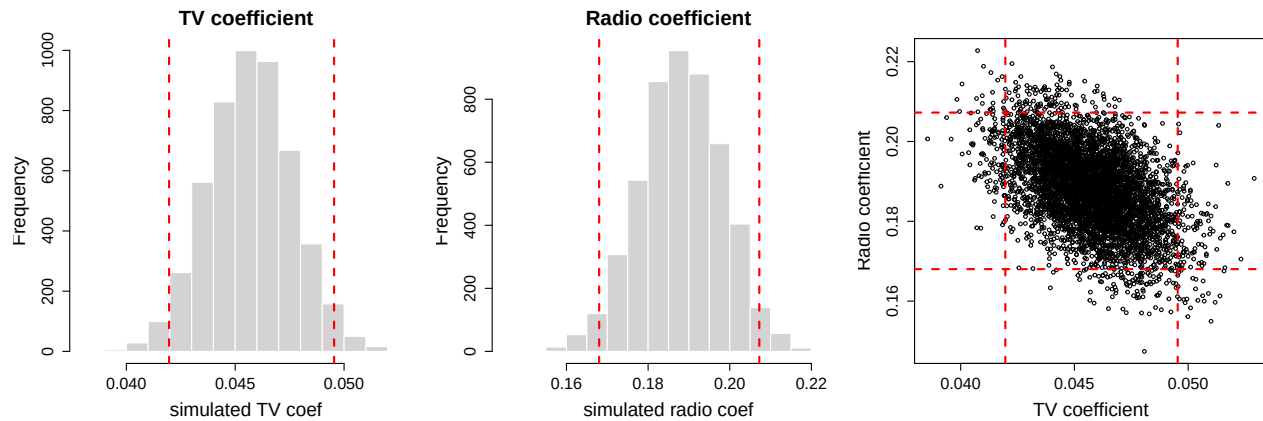
```
# Visualise the bootstrap distributions for multiple regression
par(mfrow = c(1, 3), cex.main = 1.8, cex.lab = 1.6, cex.axis = 1.4, mar = c(4, 4, 3, 1))

# Define common axis limits
tv_multi_range <- range(simcoef_multi$TV)
radio_multi_range <- range(simcoef_multi$radio)

# Histogram for the TV coefficient
hist(simcoef_multi$TV, main = "TV coefficient", xlab = "simulated TV coef",
     col = "lightgray", border = "white", breaks = 20,
     xaxt = "n") # No axis labelling at first
axis(1, at = axTicks(1)) # Explicit axis labelling
tv_multi_ticks <- axTicks(1)
abline(v = quantile(simcoef_multi$TV, c(0.025, 0.975)), col = "red", lty = 2, lwd = 2)

# Histogram for the Radio coefficient
hist(simcoef_multi$radio, main = "Radio coefficient", xlab = "simulated radio coef",
     col = "lightgray", border = "white", breaks = 20,
     xaxt = "n") # No axis labelling at first
axis(1, at = axTicks(1)) # Explicit axis labelling
radio_multi_ticks <- axTicks(1)
abline(v = quantile(simcoef_multi$radio, c(0.025, 0.975)), col = "red", lty = 2, lwd = 2)

# Scatterplot of the coefficients
plot(simcoef_multi$TV, simcoef_multi$radio, pch = 1, cex = 0.7,
     xlab = "TV coefficient", ylab = "Radio coefficient",
     xaxt = "n", yaxt = "n") # No axis labelling at first
axis(1, at = tv_multi_ticks) # Same ticks as in the TV histogram
axis(2, at = radio_multi_ticks) # Same ticks as in the Radio histogram
abline(v = quantile(simcoef_multi$TV, c(0.025, 0.975)), col = "red", lty = 2, lwd = 2)
abline(h = quantile(simcoef_multi$radio, c(0.025, 0.975)), col = "red", lty = 2, lwd = 2)
```



```
par(mfrow = c(1, 1))
```

Statistical explanation:

- The **histograms** show the distribution of the bootstrap estimates for the TV and Radio coefficients.
- The **scatterplot** shows the joint distribution of the two coefficients.
- A **correlation between the coefficients** can indicate multicollinearity, i.e., the predictors are correlated.
- In the presence of strong multicollinearity, the confidence intervals can become wider, which reduces the precision of the estimates.
- Visualising the joint distribution can help to recognise complex relationships between the coefficients that are not visible in the individual confidence intervals.

4 Inference and Significance Tests

Significance tests make it possible to assess the statistical significance of the estimated coefficients and to test hypotheses about the parameters.

4.1 t-Tests for Regression Coefficients

```
# t-tests for regression coefficients
cat("t-values and p-values for md2:\n")
```

```
## t-values and p-values for md2:
```

```
print(summary(md2)$coefficients)
```

```
##              Estimate Std. Error  t value    Pr(>|t|)
## (Intercept) 2.92109991 0.294489678  9.919193 4.565557e-19
## TV          0.04575482 0.001390356 32.908708 5.436980e-82
## radio       0.18799423 0.008039973 23.382446 9.776972e-59
```

Statistical explanation:

- The **t-test** for a regression coefficient tests the null hypothesis $H_0: \beta = 0$ against the alternative hypothesis $H_1: \beta \neq 0$.
- The **t-value** is the ratio of the estimated coefficient to its standard error: $t = \beta_{\text{hat}} / \text{SE}(\beta_{\text{hat}})$.

- The **p-value** specifies the probability of observing a t-value at least as extreme as the one computed, assuming the null hypothesis is true.
- A **small p-value** (typically < 0.05) leads to the rejection of the null hypothesis and indicates that the predictor has a significant influence on the dependent variable.
- The **standard errors** of the coefficients indicate the precision of the estimates. Smaller standard errors mean more precise estimates.
- In multiple regression, the t-tests are carried out for each coefficient, taking the other predictors in the model into account.

R explanation:

- The expression `summary(md2)$coefficients` extracts the coefficient table from the regression object.
- This table contains the estimated coefficients, their standard errors, t-values, and p-values.

4.2 F-Statistic and Model Comparison

```
# F-statistic of the overall model
f_stat <- summary(md2)$fstatistic
f_value <- f_stat["value"]
df1 <- f_stat["numdf"]
df2 <- f_stat["dendf"]
cat("\nF-statistic:", f_value, " (df1 =", df1, ", df2 =", df2, ")\n")
```

```
##
## F-statistic: 859.6177 (df1 = 2 , df2 = 197 )
```

```
# Model comparison with an additional predictor (Newspaper)
md3 <- lm(sales ~ TV + radio + newspaper, data = Advertising)
anova(md2, md3) # Comparison via F-test
```

```
## Analysis of Variance Table
##
## Model 1: sales ~ TV + radio
## Model 2: sales ~ TV + radio + newspaper
##   Res.Df    RSS Df Sum of Sq    F Pr(>F)
## 1     197 556.91
## 2     196 556.83   1  0.088717 0.0312 0.8599
```

Statistical explanation:

- The **F-statistic** tests the null hypothesis that all regression coefficients (except the intercept) are equal to zero, against the alternative hypothesis that at least one coefficient is different from zero.
- The F-statistic is defined as: $F = (ESS/p) / (RSS/(n-p-1))$, where ESS is the explained sum of squares, RSS is the residual sum of squares, n is the number of observations, and p is the number of predictors.
- A **large F-value** and a **small p-value** indicate that the model is significantly better than a model without predictors.
- The **ANOVA F-test** compares two nested models and tests whether the more complex model is significantly better than the simpler one.
- In this example, we compare a model with TV and Radio as predictors to a model that additionally includes Newspaper.
- A **non-significant F-test** ($p > 0.05$) suggests that the additional predictor (Newspaper) does not contribute significantly to the explanatory power of the model.

R explanation:

- The expression `summary(md2)$fstatistic` extracts the F-statistic from the regression object.
- The function `anova()` performs an F-test for comparing two nested models.
- The models must be nested, i.e., all predictors of the simpler model must also be contained in the more complex model.

4.3 Prediction with Confidence and Prediction Intervals

```
# Prediction: point estimate & intervals
new_values <- data.frame(TV = 230.1, radio = 37.8)
pred <- predict(md2, newdata = new_values, interval = "confidence") # Confidence
  ↪ interval
pred2 <- predict(md2, newdata = new_values, interval = "prediction") # Prediction
  ↪ interval
cat("\nConfidence interval for the expected value (mean sales):\n")
```

```
##
## Confidence interval for the expected value (mean sales):
```

```
print(pred)
```

```
##          fit          lwr          upr
## 1 20.55546 20.16278 20.94815
```

```
cat("\nPrediction interval for an individual observation:\n")
```

```
##
## Prediction interval for an individual observation:
```

```
print(pred2)
```

```
##          fit          lwr          upr
## 1 20.55546 17.21652 23.89441
```

Statistical explanation:

- The **point prediction** is the predicted value of the dependent variable for given values of the independent variables: $\hat{y} = \beta_0 + \beta_1 x_1 + \beta_2 x_2 + \dots + \beta_p x_p$
- The **confidence interval for the expected value** specifies the range in which the true expected value lies with a certain probability.
- The **prediction interval** specifies the range in which an individual new observation will lie with a certain probability.
- Prediction intervals are **wider** than confidence intervals for the expected value, because they additionally account for the variation of individual observations around the expected value.
- The width of both intervals depends on several factors:
 - Sample size: larger samples lead to narrower intervals.
 - Variability of the data: higher variability leads to wider intervals.
 - Position of the predictor values: intervals are narrowest near the mean of the predictors and become wider the further away one moves from it.

R explanation:

- The function `predict()` computes predicted values for a regression model.
- The parameter `newdata` in `predict()` specifies a data frame with the values of the predictors for which

predictions are to be made.

- The parameter `interval` in `predict()` specifies whether confidence or prediction intervals should be computed.
- The output contains three columns: `fit` (the point prediction), `lwr` (the lower bound of the interval), and `upr` (the upper bound of the interval).

4.4 Visualising Confidence and Prediction Intervals

```
# Create a grid of TV values for prediction
tv_grid <- seq(min(Advertising$TV), max(Advertising$TV), length.out = 100)
radio_mean <- mean(Advertising$radio)
newdata <- data.frame(TV = tv_grid, radio = radio_mean)

# Confidence interval for the expected value
conf_int <- predict(md2, newdata = newdata, interval = "confidence")
conf_df <- data.frame(TV = tv_grid, fit = conf_int[, "fit"],
                      lwr = conf_int[, "lwr"], upr = conf_int[, "upr"])

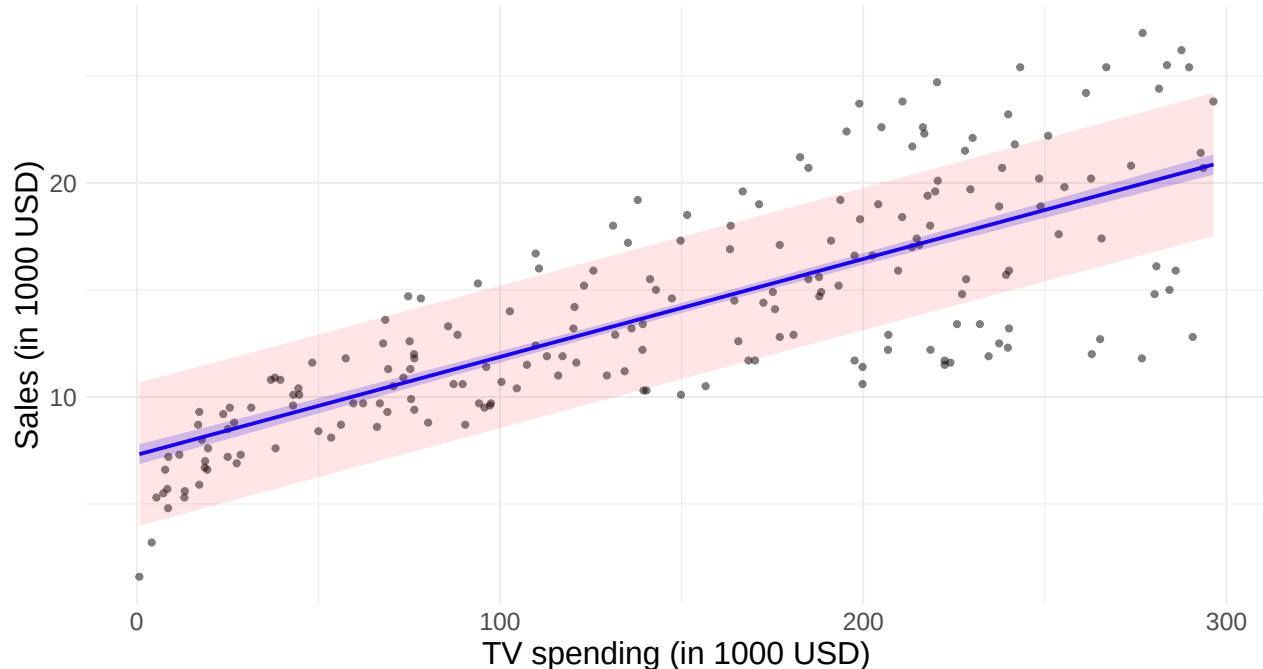
# Prediction interval
pred_int <- predict(md2, newdata = newdata, interval = "prediction")
pred_df <- data.frame(TV = tv_grid, fit = pred_int[, "fit"],
                      lwr = pred_int[, "lwr"], upr = pred_int[, "upr"])

# Visualisation
ggplot() +
  geom_point(data = Advertising, aes(x = TV, y = sales), alpha = 0.5) +
  geom_line(data = conf_df, aes(x = TV, y = fit), color = "blue", size = 1) +
  geom_ribbon(data = conf_df, aes(x = TV, ymin = lwr, ymax = upr),
            fill = "blue", alpha = 0.2) +
  geom_ribbon(data = pred_df, aes(x = TV, ymin = lwr, ymax = upr),
            fill = "red", alpha = 0.1) +
  labs(title = "Regression line with confidence and prediction intervals",
       subtitle = "Radio = mean value",
       x = "TV spending (in 1000 USD)", y = "Sales (in 1000 USD)") +
  theme_minimal() +
  theme(text = element_text(size = 16),
        axis.title = element_text(size = 18),
        axis.text = element_text(size = 14),
        plot.title = element_text(size = 20))
```

```
## Warning: Using `size` aesthetic for lines was deprecated in ggplot2 3.4.0.
## i Please use `linewidth` instead.
## This warning is displayed once per session.
## Call `lifecycle::last_lifecycle_warnings()` to see where this warning was
## generated.
```

Regression line with confidence and prediction intervals

Radio = mean value



Statistical explanation:

- The **blue line** is the regression line, showing the predicted value of **sales** for different values of **TV** when **radio** is held at its mean.
- The **blue band** is the 95% confidence interval for the expected value.
- The **red band** is the 95% prediction interval for individual observations.
- The confidence interval is **narrowest** near the mean of the predictors and becomes wider the further away one moves from it.
- The prediction interval is **considerably wider** than the confidence interval, because it additionally accounts for the variation of individual observations around the expected value.
- Most data points lie within the prediction interval, which suggests that the model describes the data well.

R explanation:

- The function `seq()` generates a sequence of values.
- The function `ggplot()` initialises a ggplot object.
- The function `geom_point()` adds points to the plot.
- The function `geom_line()` adds a line to the plot.
- The function `geom_ribbon()` adds a shaded area to the plot, defined by lower and upper bounds.
- The function `theme_minimal()` sets a minimalist theme for the plot.
- The function `theme()` adjusts various aspects of the plot, such as font sizes and styles.

5 Summary

In this lecture we have studied statistical inference in regression models. The most important concepts were:

1. **Confidence intervals for regression coefficients:** Confidence intervals quantify the uncertainty of the estimated coefficients and specify a range in which the true parameter value lies with a certain probability.

2. **Bootstrap methods:** The bootstrap method is a resampling procedure that allows us to estimate the sampling distribution of a statistic without making assumptions about the underlying distribution.
3. **Significance tests:** t-tests for individual coefficients and F-tests for the overall model or for comparing nested models make it possible to assess the statistical significance of the estimated effects.
4. **Prediction with confidence and prediction intervals:** Confidence intervals for the expected value and prediction intervals for individual observations quantify the uncertainty of the predictions.

Statistical inference is an important component of regression analysis, since it allows us to draw conclusions about the population from the sample and to assess the reliability of estimates and predictions. The bootstrap method offers a flexible alternative to classical parametric methods, in particular when the assumptions of the classical methods are not met.